HARD TRUTHS

EF Core With MongoDB vs SQL Server: What Nobody Tells You Before You Switch

While there are some similarities between the two, our currents are completely separate in substantial form.

Nagaraj Follow 6 min read · 2 days ago

Source : Nagaraj

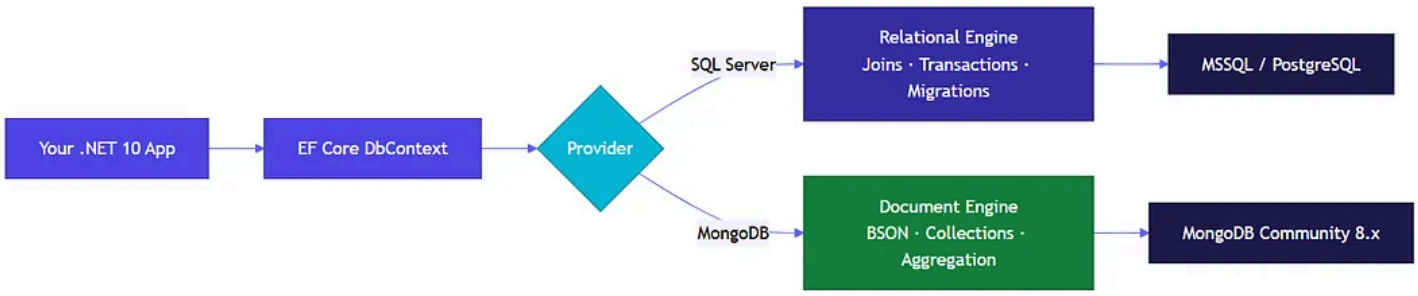
Your brain stores the movements of migrations along with LINQ queries and DbContext operations and change tracking functions as permanent skills after years of EF Core work. The announcement of a database switch to MongoDB might prompt you to think, great — EF Core has a MongoDB provider, this should be mostly the same.

It isn't. Not even close.

The provider exists. It works. The system provides an interface which functions properly yet it contains unexpected behavior that surfaces when you build systems that depend on transaction handling, joins, or memory-based data tracking.

This document functions as a vital resource which you must review before you start writing EF Core code combined with MongoDB in a production environment.

Sections



EF Core routes through the same DbContext API — but the provider changes the execution engine entirely beneath it.

The Provider That Doesn't Work Like You Think

CONCEPT

MongoDB EF Core Provider

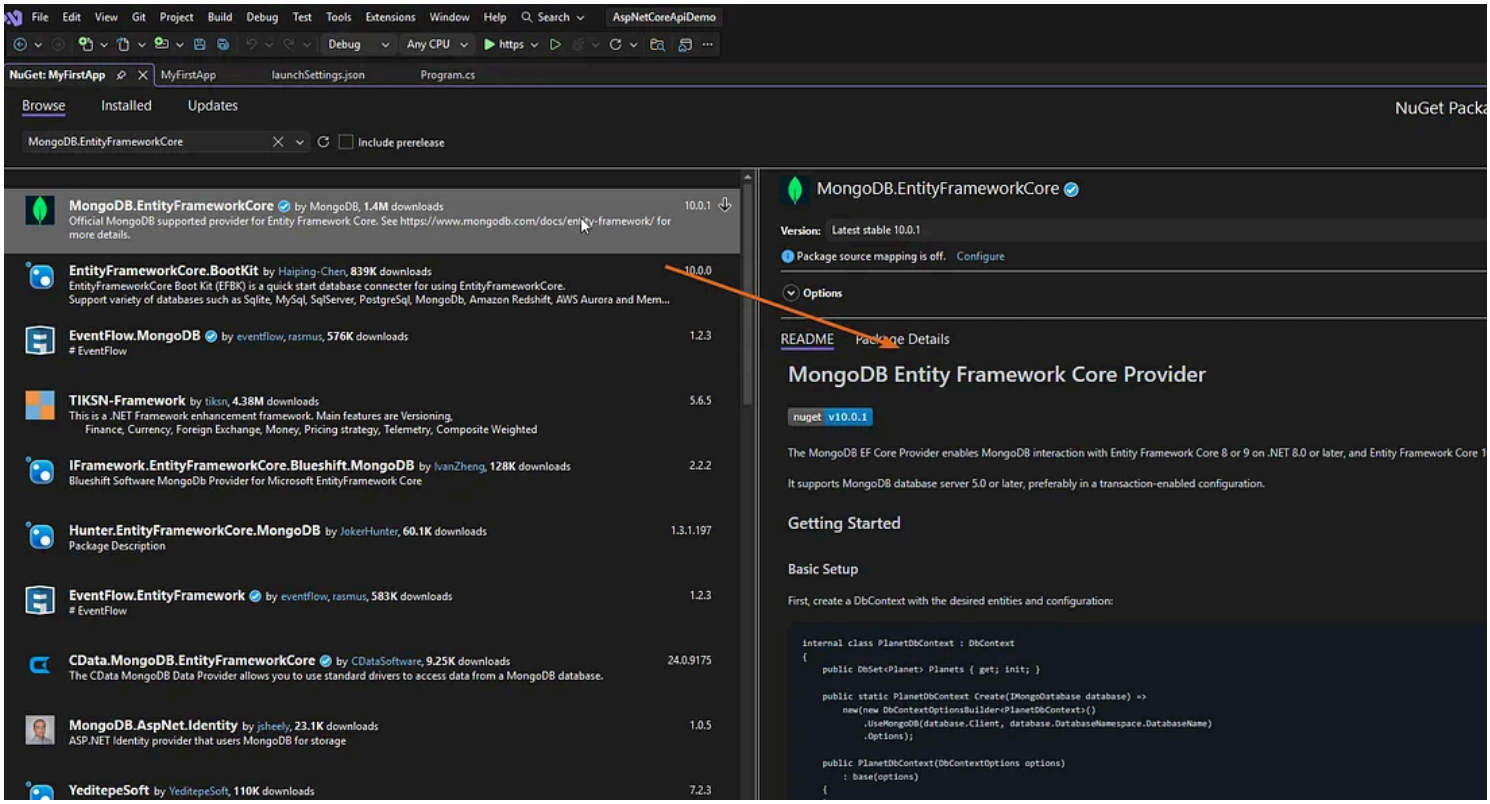
The official MongoDB package which the organization maintains provides mapping between EF Core's DbContext and LINQ API to the MongoDB document store. This package offers a different feature surface than the SQL providers.

The MongoDB EF Core provider exists as a complete system which MongoDB Inc. maintains — Microsoft does not.

The feature set requires specific domain boundaries because MongoDB does not implement relational database features that EF Core typically assumes are present.

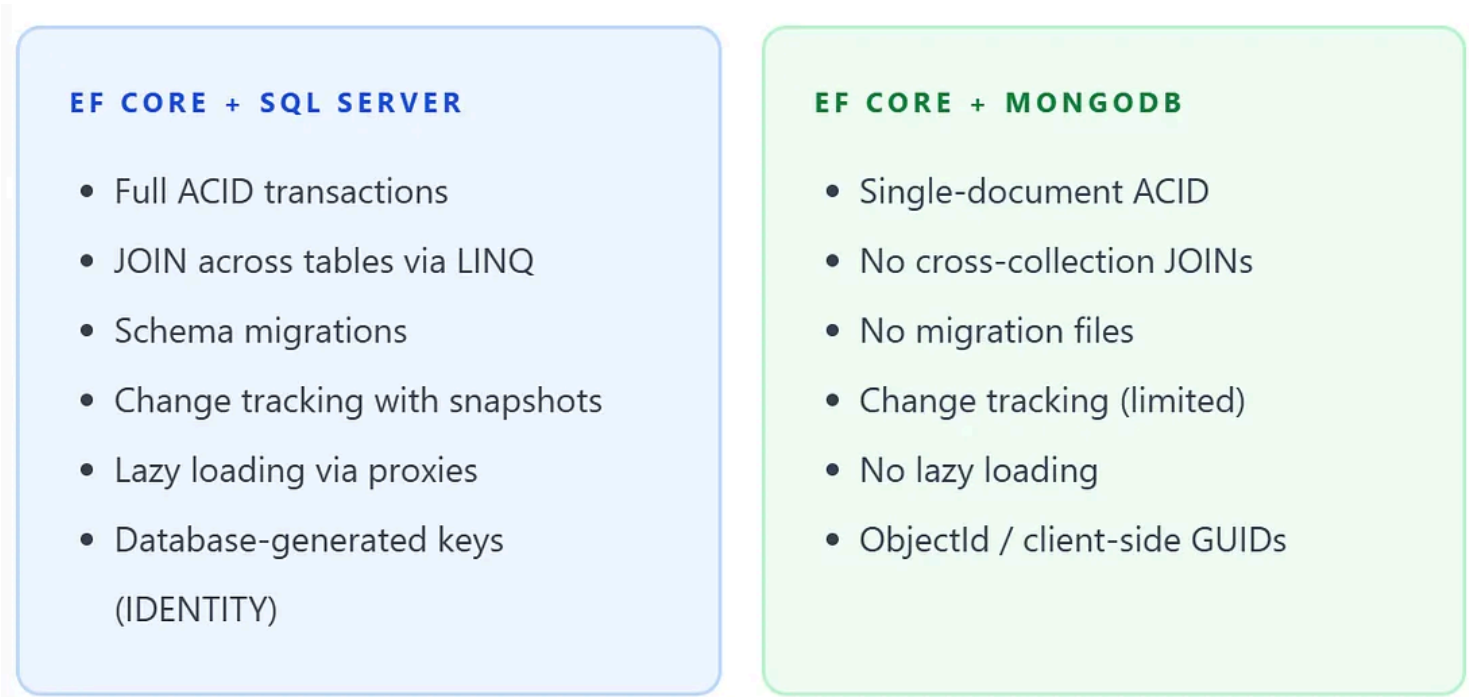
The essential package for the project is `MongoDB.EntityFrameworkCore`. The system allows you to use .NET 10 with EF Core 10 by referencing this package alongside the standard EF Core packages — there is no conflict between the two.

The provider fails to create migration SQL when you execute your first migration. The system produces no output at all. MongoDB collections operate without the schema migrations which SQL tables require.



> *"I realized that, at that time, it felt like one day it just vanished — it all goes back to some false sense of security I didn't know I had built around migrations."*

The SQL Output From EF Core That MongoDB Silently Removes



Green doesn't mean better — it means different. The MongoDB column removes features, not just renames them.

The two that bite hardest are transactions and joins. The SQL Server database requires multi-entity transaction support as its basic operational requirement. The Community Edition instance of MongoDB does not support multi-document transactions — they require either a replica set or Atlas cluster according to EF Core.

LINQ does not support cross-collection queries which function as the equivalent of a JOIN. You must use document embedding as your solution or you have to use the native MongoDB driver to access aggregation pipelines.

CONCEPT

Replica Set

The system needs multiple MongoDB instances to operate because they share the same dataset. Multi-document transactions require a replica set — even in local environments. A single standalone MongoDB instance cannot run them.

The Data Model Difference That Provides A Complete Transformation

The EF Core framework causes its most significant challenges to experienced developers when they implement this specific shift. The relational EF Core framework requires you to perform data normalization — you use multiple tables to divide your data and then use navigation properties to combine the data back into a unified structure.

The MongoDB EF Core framework requires document embedding instead. A `Customer` document contains an `Address` as a nested object instead of using a foreign key relationship to link to the Address table.

WHAT'S HAPPENING HERE

1. **No AddressId foreign key** — MongoDB does not implement referential integrity controls because all data can be stored through its embedding method.
2. **ObjectId vs int** — MongoDB generates ObjectIds client-side because the system does not have an equivalent IDENTITY column feature that creates IDs during server inserts.
3. **[BsonId] attribute** — The SQL providers of EF Core use [Key] while the MongoDB provider implements its own BSON attribute which creates two distinct conventions used within the same ORM system.

The mental model shift is: stop thinking about rows across tables, start thinking about documents that carry their own context. The customer address information should automatically be present when you retrieve a customer record — no second query required.

Your entire performance profile changes because of this. The speed of read operations increases. The process of writing data becomes more resource-intensive. Your index strategy must include nested field paths as your primary focus instead of relying on column names.

Normalization vs embedding — the same data, two completely different access patterns and storage contracts.

When MongoDB + EF Core Actually Makes Sense

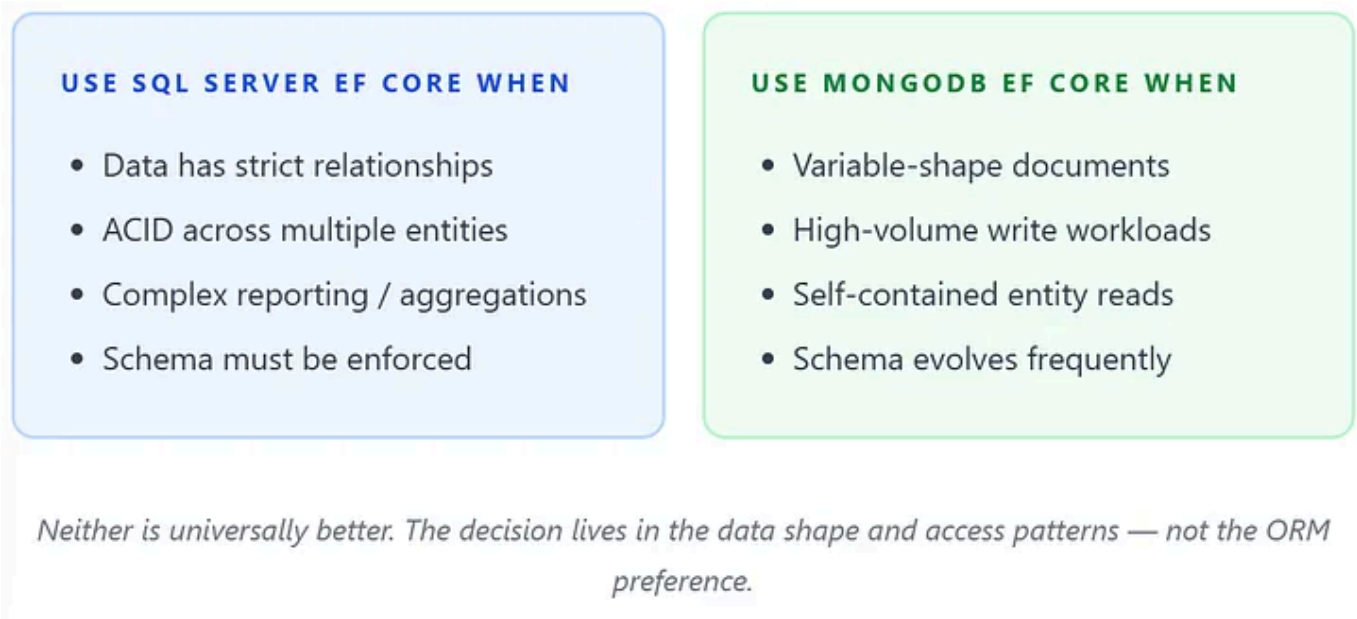
The majority of articles end their explanations at the difference. This article will proceed beyond that point. Specific situations establish MongoDB through EF Core as the best solution — and being precise about them matters.

You are storing content created by users which has unpredictable patterns of organization. The user profile contains multiple fields which differ among users — some profiles require three attributes while others need thirty. The fixed relational schema forces you to choose between permanent column utilization or ongoing migration operations. A MongoDB document stores exactly what's there.

You need to create event logs which contain complete records for each event. The system requires no joins after the initial point — users can retrieve documents through time range or event type queries which provide access to complete documents in a single fetch.

Your team already uses EF Core and doesn't want to learn the raw MongoDB driver just for one service. The EF Core provider gives you familiar LINQ

syntax at the cost of some advanced MongoDB features — that tradeoff is sometimes worth making for team velocity.



Setting It Up in .NET 10

1. **Install packages**

   Add MongoDB.EntityFrameworkCore and Microsoft.EntityFrameworkCore to your project.

2. **Configure DbContext**

   Use UseMongoDB() in OnConfiguring, passing the connection string and database name.

3. **Map your entities**

   Replace [Key] with [BsonId], use ObjectId for primary keys, embed related objects directly.

4. **Register in DI**

   Register your MongoDbContext via AddDbContext just like SQL — no different from the DI perspective.

5. **Drop migration thinking**

   There are no migrations to run. Collections are created on first write. Schema validation is handled in MongoDB Atlas or via the driver directly.

WHAT'S HAPPENING HERE

1. **UseMongoDB()** — The function operates precisely as `UseSqlServer()` while all other components of your DI configuration remain unchanged.
2. **Connection string from config** — Hardcoded MongoDB credentials should be avoided; use `IConfiguration` or environment variables for credential management.
3. **DbSet maps to a collection** — One MongoDB collection exists for each `DbSet<T>` which uses the DbSet property name as its default collection name.

Your First Integration Will Fail Because Of These Three Issues

The situation does not involve edge cases — these are the actual conditions which occur during the first week of production usage.

LINQ query translation gaps. EF Core translates LINQ to MongoDB's query language — but not all operators translate. The system experiences runtime errors because `GroupBy` requires complex projections and string functions such as `StartsWith` with case-insensitive options. Your unit tests won't catch them.

CONCEPT

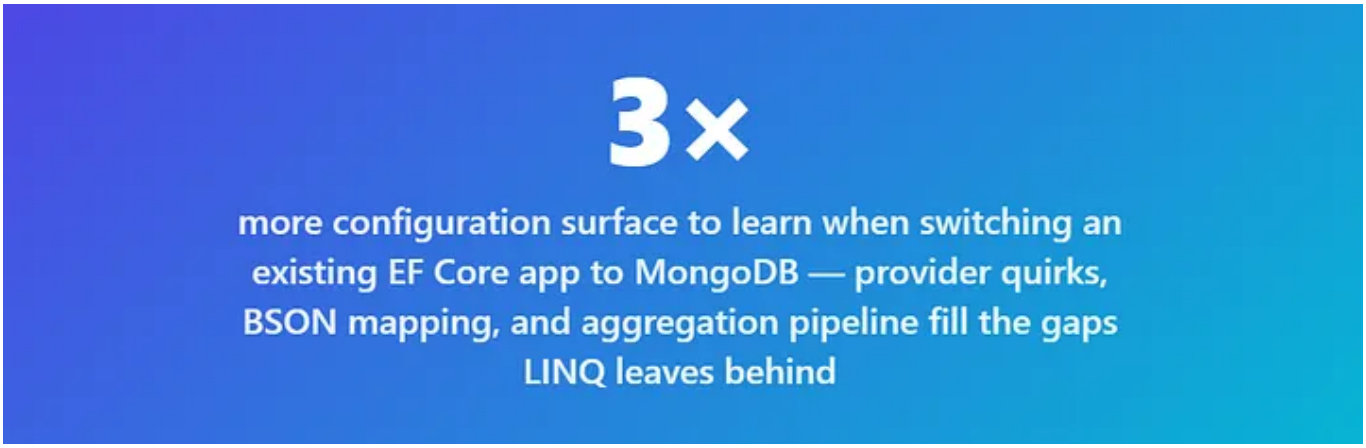
Query Translation

EF Core converts your LINQ expressions to database-specific query syntax through its conversion process which generates MQL queries for MongoDB and T-SQL queries for SQL Server. The system throws `NotSupportedException` at runtime when translation gaps are encountered.

Change tracking is limited to top-level properties. The system fails to track changes when users modify embedded objects without replacing their top-

level reference. The system will save without showing any error message yet your changes will not persist. Use `context.Entry(entity).State` to check system status during debugging.

No lazy loading at all. EF Core with SQL Server provides lazy loading support through proxy implementation. The EF Core provider for MongoDB does not support this. The system will return null when you reference a navigation property that you expected to load automatically. Your queries should explicitly specify required elements — or change your structure to use embedded elements instead of referenced ones.



QUICK REFERENCE

Multi-doc transactions	Require a replica set — will not function on a standalone Community Edition
Schema migrations	Collections are created on the first write — no migration files required
Joins via LINQ	For aggregation, embed related data or use the native driver instead
Lazy loading	Unavailable — always eager-load or embed; no proxy generation is supported
[Key] attribute	Substitute with `[BsonId]` — a different attribute convention but the same intent

The system provides access to EF Core combined with MongoDB as a legitimate solution — not a hack, not a workaround. The application functions as a different software system which uses a well-known interface. The question isn't whether you can use EF Core's API. The question is whether you're willing to unlearn the SQL assumptions baked into how you model data.

The organization should maintain its use of SQL Server when its data needs relational database management. The MongoDB EF Core provider in .NET 10 provides an adequate solution for teams which operate with document-based data and understand the limitations of this technology.

Thank you for reading!

👏👏👏

Hit the applause button and show your love❤️, and please follow➡️ for a lot more similar content! Let's keep the good vibes flowing!

Dotnet Mongodb Entity Framework Core Csharp Backend

Published in Towards Dev

10K followers · Last published 6 hours ago

Follow

A publication for sharing projects, ideas, codes, and new theories.

Written by Nagaraj

874 followers · 46 following

Follow

Full Stack Dev | UI/UX | Clean code, AI enthusiast, clear thinking, and real-world lessons to help you grow as a developer.